Experiment 'p_5_7_count_words' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_5_7_count_words
Natural language statement: null
Method signature: p_5_7_count_words(str: string) returns (word_count: int)

Ensures

- `word_count == count_words(str)`

Functional Code Given

```
function remove_leading_spaces(s: string): (result: string)
    ensures |s| > 0 && s[0] == ' ' ==> |result| < |s|
{
    if |s| == 0 then s
    else if s[0] == ' ' then remove_leading_spaces(s[1..])
    else s
}

function remove_word(s: string): (result: string)
    ensures |s| > 0 && s[0] != ' ' ==> |result| < |s|
{
    if |s| == 0 then s
    else if s[0] != ' ' then remove_word(s[1..])
    else s
}

function count_words(s: string): int
    decreases |s|
{
    var s2 := remove_leading_spaces(s);
```

```
    if |s2| > 0 then 1 + count_words(remove_word(s2))
    else 0
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_5_7_count_words(str: string) returns (word_count: int)

The method should respect the following contract:

ensures word_count == count_words(str)

The contract uses the following dafny code:

function remove_leading_spaces(s: string): (result: string)
    ensures |s| > 0 && s[0] == ' ' ==> |result| < |s|
{
    if |s| == 0 then s
    else if s[0] == ' ' then remove_leading_spaces(s[1..])
    else s
}

function remove_word(s: string): (result: string)
    ensures |s| > 0 && s[0] != ' ' ==> |result| < |s|
{
    if |s| == 0 then s
    else if s[0] != ' ' then remove_word(s[1..])
    else s
}

function count_words(s: string): int
    decreases |s|
{
    var s2 := remove_leading_spaces(s);
    if |s2| > 0 then 1 + count_words(remove_word(s2))
    else 0
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnR6SpMhRsU1gzQmrWrNi2P7wLFcH
Model: gpt-5.5-2026-04-23
Created at: 1780674628
Finish reason: stop
Usage: 692 tokens in, and 4941 tokens out

```
{
  word_count := 0;
  var i := 0;
  assert str[0..] == str;

  while i < |str|
    invariant 0 <= i <= |str|
    invariant 0 <= word_count
    invariant word_count + count_words(str[i..]) == count_words(str)
  {
    if str[i] == ' ' {
      assert 0 <= i < |str|;
      assert |str[i..]| > 0;
      assert (str[i..])[0] == ' ';
      assert (str[i..])[1..] == str[i + 1..];
      assert remove_leading_spaces(str[i..]) == remove_leading_spaces((str[i..])[1..]);
      assert remove_leading_spaces(str[i..]) == remove_leading_spaces(str[i + 1..]);
      assert count_words(str[i..]) == count_words(str[i + 1..]);
      assert word_count + count_words(str[i + 1..]) == count_words(str);
      i := i + 1;
    } else {
      var start := i;
      var old_count := word_count;
      assert 0 <= start < |str|;
      assert str[start] != ' ';
      assert |str[start..]| > 0;
      assert (str[start..])[0] != ' ';
      assert remove_leading_spaces(str[start..]) == str[start..];
      assert count_words(str[start..]) == 1 + count_words(remove_word(str[start..]));
      assert old_count + count_words(str[start..]) == count_words(str);
      word_count := old_count + 1;
      assert word_count + count_words(remove_word(str[start..])) == count_words(str);
      assert i == start;
      assert word_count + count_words(remove_word(str[i..])) == count_words(str);

      while i < |str| && str[i] != ' '
        invariant start <= i <= |str|
        invariant 0 <= word_count
        invariant word_count + count_words(remove_word(str[i..])) == count_words(str)
      {
        assert 0 <= i < |str|;
        assert |str[i..]| > 0;
        assert (str[i..])[0] != ' ';
```

```
      assert (str[i..])[1..] == str[i + 1..];
      assert remove_word(str[i..]) == remove_word((str[i..])[1..]);
      assert remove_word(str[i..]) == remove_word(str[i + 1..]);
      assert word_count + count_words(remove_word(str[i + 1..])) == count_words(str);
      i := i + 1;
    }

    if i == |str| {
      assert str[i..] == "";
      assert remove_word(str[i..]) == str[i..];
    } else {
      assert i < |str|;
      assert str[i] == ' ';
      assert |str[i..]| > 0;
      assert (str[i..])[0] == ' ';
      assert remove_word(str[i..]) == str[i..];
    }
    assert word_count + count_words(str[i..]) == count_words(str);
  }
 }

 assert i == |str|;
 assert str[i..] == "";
 assert count_words(str[i..]) == 0;
 assert word_count == count_words(str);
}
```

Final Program

```
function remove_leading_spaces(s: string): (result: string)
   ensures |s| > 0 && s[0] == ' ' ==> |result| < |s|
{
   if |s| == 0 then s
   else if s[0] == ' ' then remove_leading_spaces(s[1..])
   else s
}

function remove_word(s: string): (result: string)
   ensures |s| > 0 && s[0] != ' ' ==> |result| < |s|
{
   if |s| == 0 then s
   else if s[0] != ' ' then remove_word(s[1..])
   else s
}

function count_words(s: string): int
   decreases |s|
{
   var s2 := remove_leading_spaces(s);
   if |s2| > 0 then 1 + count_words(remove_word(s2))
```

```
    else 0
}

method p_5_7_count_words(str: string) returns (word_count: int)
       ensures word_count == count_words(str)
{
  word_count := 0;
  var i := 0;
  assert str[0..] == str;

  while i < |str|
    invariant 0 <= i <= |str|
    invariant 0 <= word_count
    invariant word_count + count_words(str[i..]) == count_words(str)
  {
    if str[i] == ' ' {
      assert 0 <= i < |str|;
      assert |str[i..]| > 0;
      assert (str[i..])[0] == ' ';
      assert (str[i..])[1..] == str[i + 1..];
      assert remove_leading_spaces(str[i..]) == remove_leading_spaces((str[i..])[1..]);
      assert remove_leading_spaces(str[i..]) == remove_leading_spaces(str[i + 1..]);
      assert count_words(str[i..]) == count_words(str[i + 1..]);
      assert word_count + count_words(str[i + 1..]) == count_words(str);
      i := i + 1;
    } else {
      var start := i;
      var old_count := word_count;
      assert 0 <= start < |str|;
      assert str[start] != ' ';
      assert |str[start..]| > 0;
      assert (str[start..])[0] != ' ';
      assert remove_leading_spaces(str[start..]) == str[start..];
      assert count_words(str[start..]) == 1 + count_words(remove_word(str[start..]));
      assert old_count + count_words(str[start..]) == count_words(str);
      word_count := old_count + 1;
      assert word_count + count_words(remove_word(str[start..])) == count_words(str);
      assert i == start;
      assert word_count + count_words(remove_word(str[i..])) == count_words(str);

      while i < |str| && str[i] != ' '
        invariant start <= i <= |str|
        invariant 0 <= word_count
        invariant word_count + count_words(remove_word(str[i..])) == count_words(str)
      {
        assert 0 <= i < |str|;
        assert |str[i..]| > 0;
        assert (str[i..])[0] != ' ';
        assert (str[i..])[1..] == str[i + 1..];
        assert remove_word(str[i..]) == remove_word((str[i..])[1..]);
        assert remove_word(str[i..]) == remove_word(str[i + 1..]);
```

```
      assert word_count + count_words(remove_word(str[i + 1..])) == count_words(str);
      i := i + 1;
    }

    if i == |str| {
      assert str[i..] == "";
      assert remove_word(str[i..]) == str[i..];
    } else {
      assert i < |str|;
      assert str[i] == ' ';
      assert |str[i..]| > 0;
      assert (str[i..])[0] == ' ';
      assert remove_word(str[i..]) == str[i..];
    }
    assert word_count + count_words(str[i..]) == count_words(str);
  }
 }

 assert i == |str|;
 assert str[i..] == "";
 assert count_words(str[i..]) == 0;
 assert word_count == count_words(str);
}
```

Total Token Usage

Input tokens: 692
Output tokens: 4941
Reasoning tokens: 4096
Sum of 'total tokens': 5633

Experiment Timings

Overall Experiment started at 1780674628296, ended at 1780674760006, lasting 131710ms (131.71 seconds)
Iteration #1 started at 1780674628297, ended at 1780674760006, lasting 131709ms (131.71 seconds)